241080076 Vishwajeet Waghmare | 241080077 Awwab Wadekar
241080078 Yash Wadhwa | 241080082 Abiraj Kangotra

Hospital Management System (HMS)

1.PROBLEM STATEMENT

1. Introduction

> **"A lightweight, offline-first Windows desktop application paired with embedded display clients that streamlines core hospital workflows: patient queueing, doctor schedules, bed management, and pharmacy inventory, with local zero-latency updates and automated, periodic cloud synchronization."**

2. Existing Systems

Modern healthcare facilities typically rely on three main categories of management solutions:

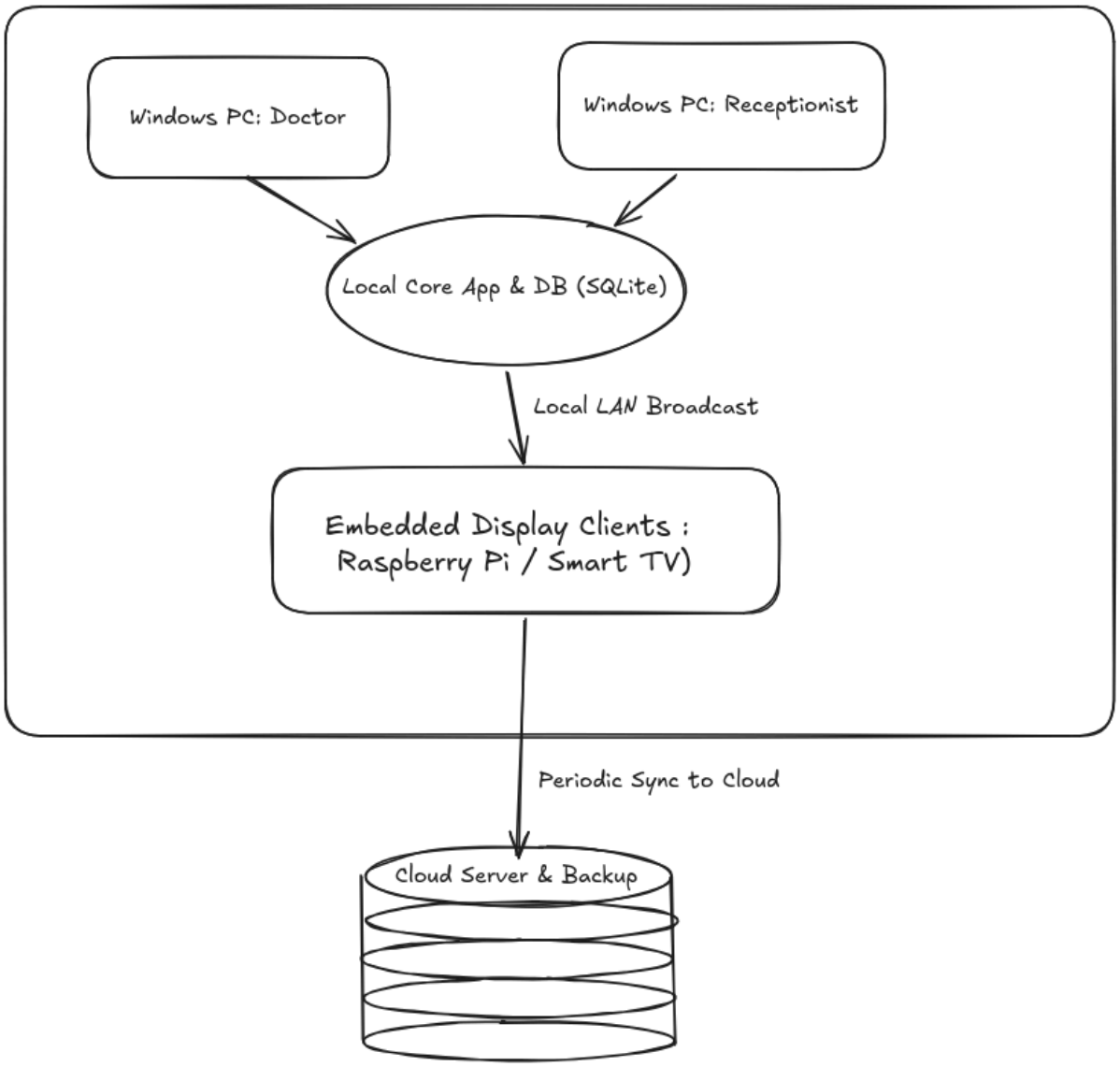
- **Cloud-Only Web SaaS Platforms:** Browser-based Enterprise Hospital Information Systems (HIS) where all data entry, retrieval, and state changes happen over cloud servers (e.g., standard web portals).
- **Heavy On-Premise Enterprise Systems:** Legacy client-server systems deployed on local, high-maintenance physical servers within the hospital data center.
- **Manual & Fragmented Solutions:** Paper registers, whiteboards, and standalone Excel spreadsheets commonly used in small-to-medium clinics or peripheral hospital departments.

3. Weaknesses of Existing Systems

- **Vulnerability to Network Outages (Cloud Systems):** Web-based platforms freeze or become completely inaccessible during internet drops, ISP throttling, or server maintenance—paralyzing reception desks and emergency triage.
- **High Latency and UI Friction:** Web forms require constant server round-trips to validate inputs. Receptionists lose critical seconds per patient navigating bloated, multi-nested browser menus.
- **Complex Infrastructure & High Cost (Enterprise On-Prem):** On-premise enterprise setups require dedicated database administrators, server hardware maintenance, and complex networking, making them unaffordable for smaller hospitals.
- **Decoupled Display Systems:** Existing platforms rarely integrate seamlessly with physical display boards in waiting areas. Hospitals often resort to manually editing digital signboards or running separate third-party signage software.
- **Data Entry Fatigue for Medical Personnel:** Over-engineered interfaces force doctors to click through dozens of irrelevant tabs just to check bed status or patient arrival times.

4. How We Will Tackle Them

We propose a **hybrid, offline-first architecture** designed specifically for high-reliability local operations with periodic background cloud synchronization.



- **Local-First Execution:** All daily actions (patient check-in, doctor status updates, medicine queries, bed assignments) write directly to a ultra-fast local database on the primary workstation PC. The system operates at **100% functionality with zero internet**.
- **Background Cloud Sync Engine:** A non-blocking background worker periodically batches and syncs local transactions to the cloud when internet is available. This provides off-site data backup and remote access without tying operational availability to the network.
- **Tailored Native Windows Executable (.exe):** A standalone desktop application built with keyboard shortcuts, auto-suggest inputs, and streamlined views designed for high-speed data entry by receptionists and quick lookups by doctors.

- **Direct Local LAN Display Streaming:** embedded hardware nodes (such as low-cost display clients driving waiting-room screens) fetch status updates directly over the hospital's local Wi-Fi/Ethernet from the desktop application, ensuring instantaneous queue updates without hitting external APIs.

5. What is Different in Our Approach?

Feature / Metric	**Traditional Cloud HMS**	**Our Hybrid Approach**
Internet Dependency	Hard requirement (App breaks without internet)	**Zero dependency for daily operations**
Data Entry Latency	High (Server round-trips over Web)	**Instant (<10ms local response time)**
Deployment Complexity	Complex cloud setups or on-site server racks	**Single .exe installer on a standard Windows PC**
Display Integration	Manual web updates or third-party digital signage	**Native LAN broadcasting to embedded display clients**
Cost Profile	High recurring subscription or server maintenance	**Low local footprint with minimal cloud bandwidth costs**

6. Limitations and Technologies

Technologies Stack

- **Desktop PC Application (Windows):**
  - *Language/Framework:* **C# (.NET / WPF)** or **Python (PyQt6 / CustomTkinter)** compiled into a standalone .exe.
  - *Local Database:* **SQLite** or **LiteDB** (Zero-configuration embedded file database).
- **Embedded Display System:**
  - *Hardware:* **Raspberry Pi** or **ESP32** connected to HDMI displays / LED matrices.

    - *Software:* Lightweight **Python (Tkinter/Pygame)** or **Web-based Kiosk Mode (Electron/Lite browser)** listening to local network events via WebSockets or MQTT.
- **Cloud Sync Layer:**
    - *Backend API:* **Python (FastAPI)** or **Node.js** hosted on lightweight cloud infrastructure (AWS/Supabase/Firebase).
    - *Database:* **PostgreSQL** for remote backup and multi-branch aggregation.

Anticipated Limitations

- **Sync Conflict Handling:** If multiple offline devices update the same bed or medicine stock simultaneously, conflict resolution rules (such as timestamp-based *Last-Write-Wins* or append-only transactional logs) must be strictly enforced.
- **Initial Local Area Network (LAN) Setup:** Embedded displays require a simple, stable router/LAN connection within the facility to discover and pair with the main desktop application.
- **Hardware Storage Management:** The primary host PC must periodically prune or archive old historical logs to keep local database size compact over years of usage.